CONCLUSÃO DE CURSO

Aplicação de Ferramentas de Automação (Netmiko, NAPALM e Jinja2) na Configuração de Roteadores Cisco em Ambiente Simulado de Provedor de Serviços de Internet

Gabriel Henrique de Salles

Orientador: Prof. Luis Jose Rohling

Avaliador: Prof. Ricardo Cassiano Nabhen

Pós-Graduação em Engenharia de Redes e Telecomunicações · PUCPR · 24/10/2026

Ativar um cliente ainda é trabalho manual na CLI

No PROVEDOR, o técnico acessa o roteador de borda por SSH, digita os comandos do procedimento, confere e registra.



Tempo

Minutos de trabalho especializado por ativação



Erros silenciosos

Comando válido com valor errado é aceito sem aviso



Verificação

Depende de lembrar onde e como conferir



Rastreabilidade

Quem mudou o quê, e quando, fica disperso



Escala

Esforço cresce junto com o número de clientes

Aconteceu na montagem deste laboratório: ao configurar o OSPF à mão em 4 roteadores, a rede de gerência da borda 2 foi declarada na borda 3. O erro só apareceu na tabela de rotas do core.

Objetivo: aplicar ferramentas existentes e medir o ganho

Aplicar Netmiko, NAPALM e Jinja2 à ativação de clientes corporativos em roteadores Cisco, num laboratório com a arquitetura de um ISP, e medir o ganho em relação ao processo manual.

Linha de Projeto: ferramentas de código aberto, não uma ferramenta nova.

Escopo: IP/BGP, em laboratório (CML).

ID	Pergunta	Como foi medido
D1	É mais rápido e erra menos?	10 repetições: manual × script
D2	Detecta dados errados e falhas?	7 erros injetados
D3	Escala com mais clientes?	Lotes de 1, 5 e 10 (48 ativações)
D4	Mantém a rede padronizada?	3 rodadas, desvios sorteados às cegas
D5	E se uma borda falhar?	edge-1 isolada, 5 clientes

Médias e medianas em todos os tempos; registros completos no Apêndice A.

Quatro peças, cada uma com um papel



YAML

Fonte da verdade

Dados dos clientes e inventário dos roteadores, versionados em Git



Jinja2

Gera a configuração

Modelo escrito uma vez; os dados preenchem as lacunas {{ }}



Netmiko

Aplica e verifica

Sessão SSH na CLI: envia comandos e consulta o resultado



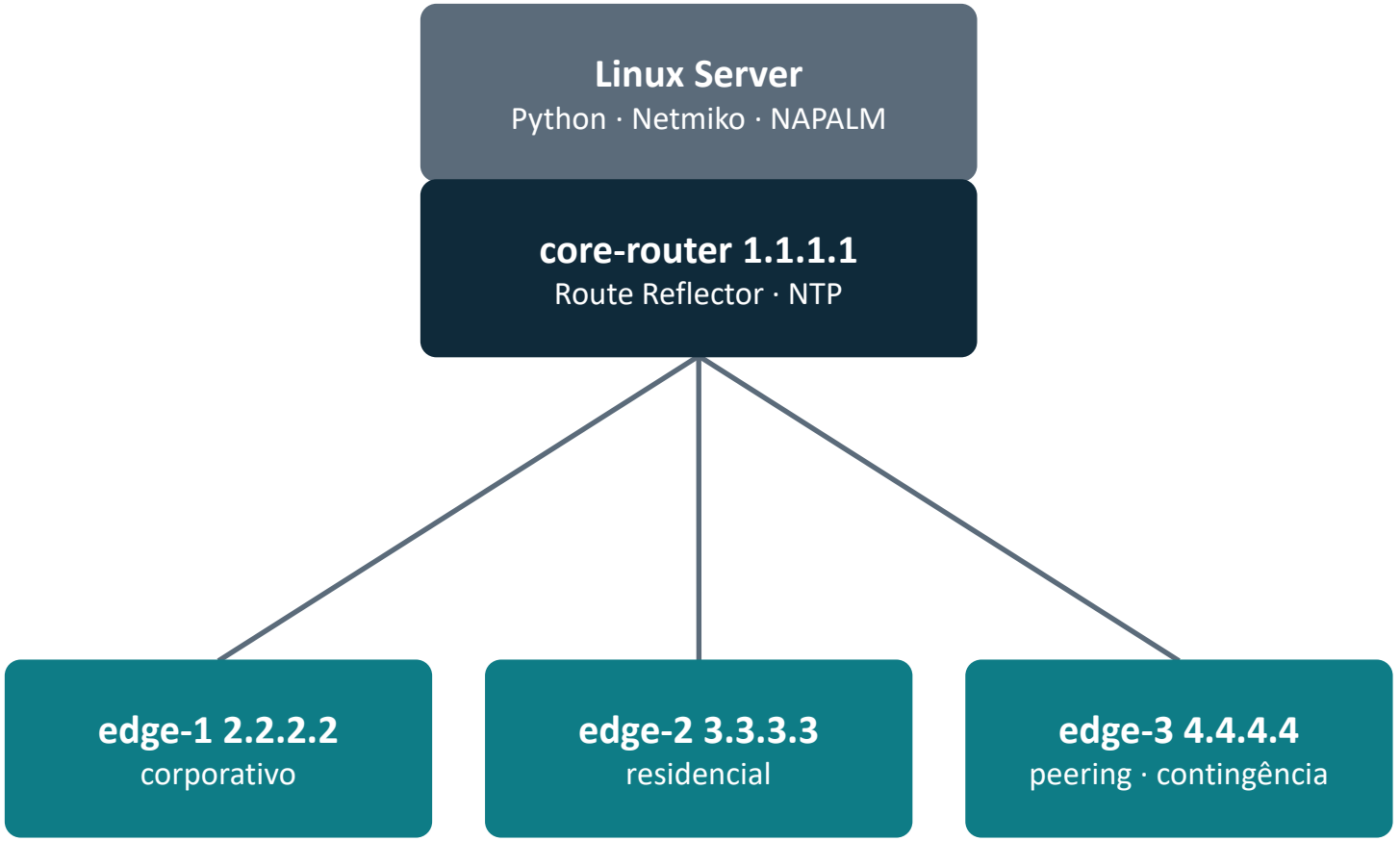
NAPALM

Lê de forma estruturada

Inventário (get_facts), backup e auditoria (get_config)

Por que não Ansible ou Nornir? Custo zero, funciona no parque existente via SSH e deixa cada etapa visível no código, o que permite medi-las separadamente. O Nornir usa estas mesmas bibliotecas: é a evolução natural do fluxo.

Laboratório: core e borda de um ISP em escala reduzida



Clientes = Loopback1xx · 172.16.ID.0/24 anunciado no BGP

Cisco Modeling Labs 2.10 Free

limite de 5 nós

IOL-XE · IOS XE 17.18.2

OSPF área 0

só a infraestrutura (loopbacks)

iBGP AS 65001

prefixos dos clientes, via Route Reflector

Rastreabilidade

NTP no core, log de cada comando, Git

Fluxo de ativação em cinco etapas

1

Validação

Campos, prefixo e roteador conferidos, sem tocar na rede

2

Pré-check

Prefixo já em uso? Cliente já ativo?

3

Gerar e aplicar

Jinja2 monta os comandos; Netmiko envia à borda

4

Verificar

No Route Reflector: prefixo no BGP e ping ≥ 80%

5

Decidir

Passou: salva. Falhou: rollback automático

Entrada (YAML)

```
id: 101, nome: EMPRESA-ALFA, circuito: CORP-0101,
velocidade: 100M, roteador: edge-router-1,
prefixo: 172.16.101.0/24
```

Saída (gerada pelo Jinja2)

```
interface Loopback101
 description CLIENTE:EMPRESA-ALFA | CIRCUITO:CORP-
0101 | VEL:100M
router bgp 65001 / network 172.16.101.0 mask
255.255.255.0
```

D1 · Proposta: manual × automatizado

D1

PERGUNTA

A automação é mais rápida e erra menos que o processo manual?

Como foi feito

1. O mesmo cliente é ativado à mão, digitado a partir de uma ficha
2. Cronômetro do início da configuração até a verificação OK no core
3. Em seguida, o mesmo cliente é ativado pelo script (03_ativar_cliente.py)
4. 10 repetições pareadas: R01–R05 na edge-1 e R06–R10 na edge-2

O que foi medido

Tempo (média e mediana) | **Erros** | **Configuração final**

D1 · Resultado: de 162,0 s para 5,7 s

−96,5%

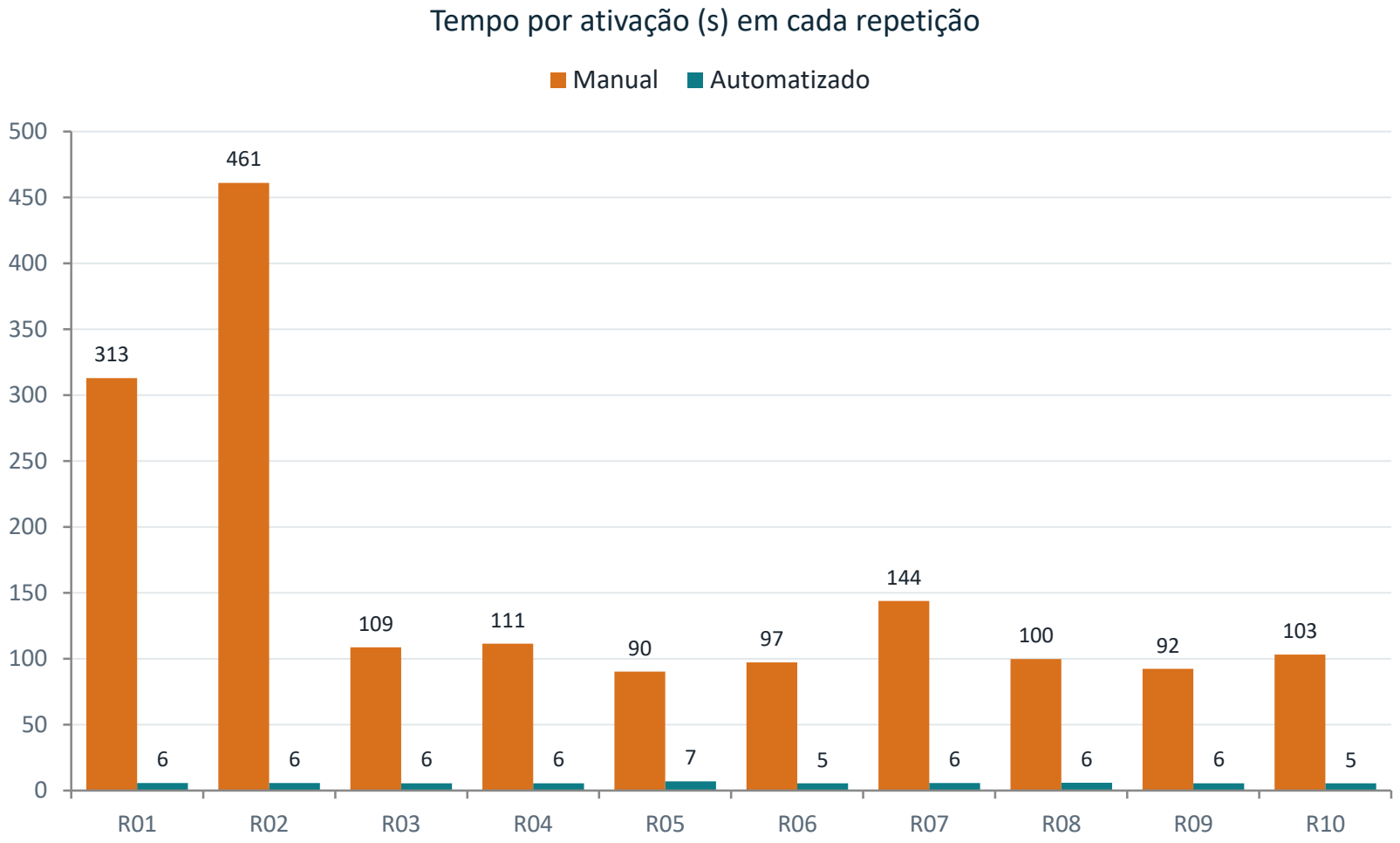
no tempo médio (mediana: −94,7%)

5 × 0

erros: manual × automatizado

0,4 s

desvio padrão automático (manual: 124,5 s)



D2 · Proposta: erros injetados

D2

PERGUNTA

O fluxo detecta dados errados e falhas de rede?

Como foi feito

1. 7 cenários de erro, aplicados um de cada vez
2. 4 erros de dados: octeto inválido, bits de host, roteador inexistente, campo ausente
3. 2 conflitos com a rede: prefixo já usado e cliente ativado duas vezes
4. 1 falha de rede: sessão BGP da borda derrubada antes da ativação

O que foi medido

Etapa que barrou | **Tempo** | **Sobrou configuração?**

D2 · Resultado: 7 de 7 erros barrados

ID	Erro injetado	Barrado em	Tempo
E1	Prefixo com octeto inválido	Validação	0,00 s
E2	Prefixo com bits de host	Validação	0,00 s
E3	Roteador inexistente	Validação	0,00 s
E4	Prefixo já usado por outro cliente	Pré-check	1,11 s
E5	Campo obrigatório ausente	Validação	0,00 s
E6	Sessão BGP da borda derrubada	Verificação + rollback	51,96 s
E7	Cliente ativado duas vezes	Pré-check	1,11 s

7/7

4 barrados antes de acessar a rede

2 barrados no pré-check

1 detectado e desfeito sozinho

Nenhuma configuração incorreta ficou nos roteadores.

D3 · Proposta: escala em lotes

D3

PERGUNTA

O fluxo escala com o número de clientes?

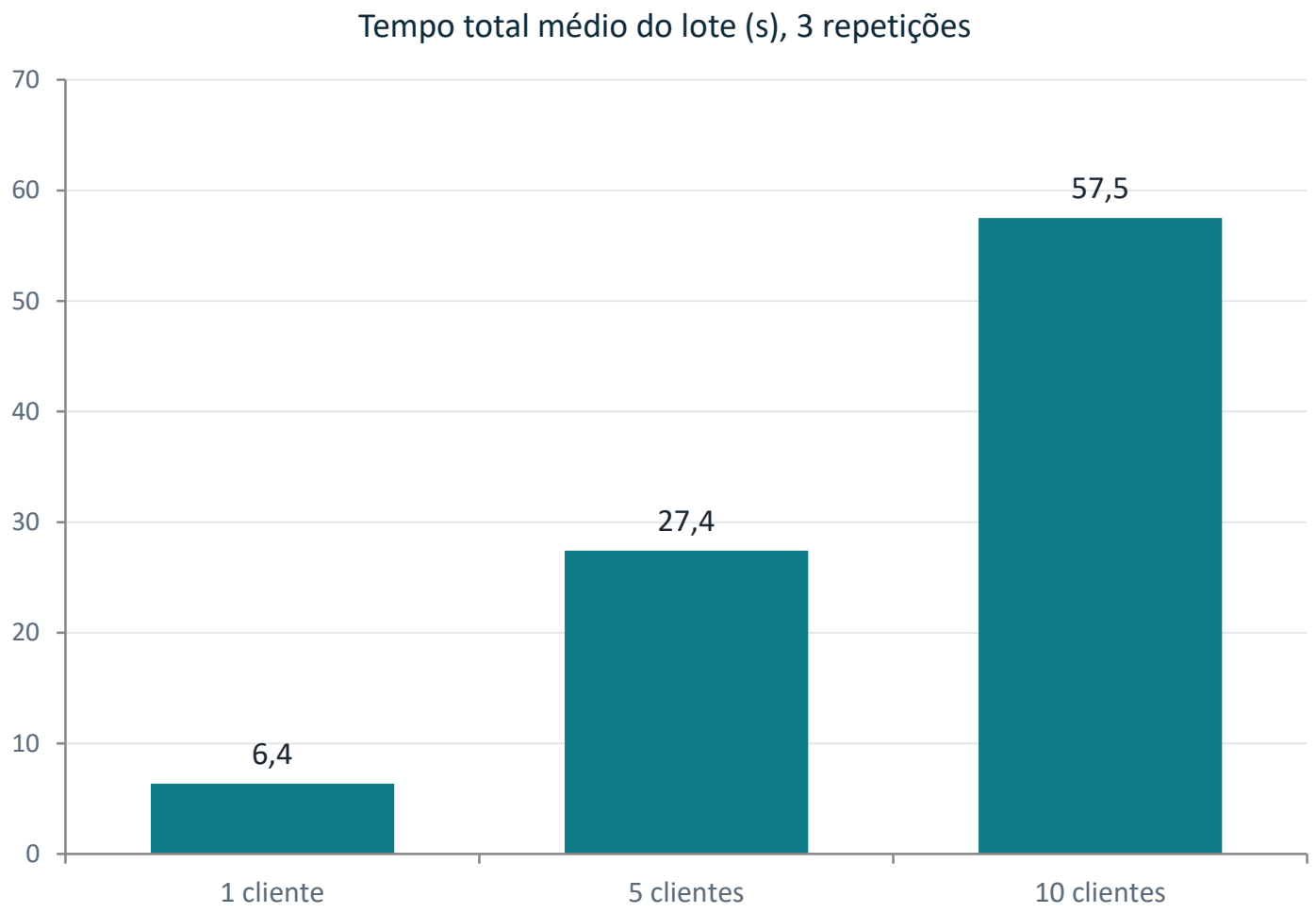
Como foi feito

1. Lotes de 1, 5 e 10 clientes, ativados em sequência pelo script
2. 3 repetições de cada tamanho: 9 lotes, 48 ativações
3. Cada cliente passa pelo fluxo completo de 5 etapas, com verificação no RR

O que foi medido

- **Tempo do lote**
- **Tempo por cliente**
- **Ajuste linear (R^2)**

D3 · Resultado: ≈ 5,7 s por cliente, linear



48/48

ativações concluídas em 9 lotes

R² 0,97

ajuste linear do tempo total

≈ 9,5 min

projeção para 100 ativações

D4 · Proposta: auditoria de conformidade

D4

PERGUNTA

A automação ajuda a manter a rede padronizada?

Como foi feito

1. Auditoria de base: a rede começa conforme o padrão (8 regras + 1 aviso)
2. O script sorteia 2 a 4 desvios entre 20 combinações, sem revelar quais
3. Auditoria manual cronometrada; achados registrados no terminal
4. Auditoria automatizada: NAPALM lê os 4 roteadores e aplica as regras
5. Gabarito revelado, comparação e reversão. 3 rodadas (piloto descartado)

O que foi medido

Desvios detectados | **Falsos positivos** | **Tempo**

D4 · As regras do padrão auditadas

ID	Regra	Por que importa	Onde	Desvio injetado?
C01	no ip domain lookup	Erro de digitação não trava o terminal buscando DNS	Todos	**Sim**
C02	logging buffered 64000	Eventos guardados no roteador para investigar falhas	Todos	**Sim**
C03	Log com data e milissegundos	Comparar registros de equipamentos diferentes	Todos	**Sim**
C04	Usuário admin com privilégio 15	Acesso administrativo padronizado (usado pelo script)	Todos	**Não**
C05	Acesso remoto com login local	Todo acesso remoto exige usuário e senha	Todos	**Não**
C06	Acesso remoto somente por SSH	Telnet trafega a senha em texto aberto	Todos	**Sim**
C07	Core como servidor NTP	Horário único em toda a rede	Core	**Sim**
C08	Descrição padrão nas interfaces de cliente	Identifica cliente, circuito e velocidade	Bordas	**Sim**
A01	Servidor HTTP desabilitado (aviso)	Recomendação: menos portas abertas	Todos	—

32 verificações por auditoria (8 por roteador). C04 e C05 são auditadas, mas não viram desvio: cortariam o acesso do próprio script. A01 é só aviso.

D4 · Resultado: 4 s contra quase 6 minutos

Manual (operador)

7 de 9 · 346,1 s

78% dos desvios · mediana 371,2 s · 0 falsos positivos

Automatizada (NAPALM)

9 de 9 · 4,3 s

100% dos desvios · 32 verificações · 0 falsos positivos

Rodada	Desvios	Manual	Automatizado
1	3	2 de 3 · 371,2 s	3 de 3 · 4,4 s
2	4	3 de 4 · 405,6 s	4 de 4 · 4,2 s
3	2	2 de 2 · 261,5 s	2 de 2 · 4,3 s

Os 2 desvios perdidos estavam na tela: loopback sem descrição e Telnet liberado no core.

Achado real: ip http server habilitado nos 4 roteadores (aviso A01).

D5 · Proposta: falha de uma borda

D5

PERGUNTA

O que o fluxo faz se a borda do cliente estiver fora do ar?

Como foi feito

1. 5 clientes: 2 com acesso redundante (edge-1 e edge-3), 1 só na edge-1, 2 na edge-2
2. Controle: os 5 ativados com a rede normal
3. Falha: edge-1 isolada (interface do core desligada)
4. Antes de configurar, o script testa se o roteador principal está no ar (SSH, espera até 5 s)
5. Decide pelo cadastro: principal, contingência ou falha segura

O que foi medido

Decisão por cliente | **Tempo de detecção** | **Configuração parcial?**

D5 · Resultado: decisão certa por cliente

Cliente	Acesso	Rede normal (controle)	Edge-1 isolada
111	Redundante (edge-1 e edge-3)	Principal · OK · 3,75 s	**Contingência na edge-3 · OK · 3,72 s**
112	Redundante (edge-1 e edge-3)	Principal · OK · 3,40 s	**Contingência na edge-3 · OK · 3,66 s**
113	Circuito único (edge-1)	Principal · OK · 3,38 s	**Falha segura: nada aplicado**
114	Circuito único (edge-2)	Principal · OK · 3,34 s	Principal · OK · 6,87 s
115	Circuito único (edge-2)	Principal · OK · 3,76 s	Principal · OK · 6,71 s



Detecção em 0 s
o core respondeu destino inalcançável (ICMP)

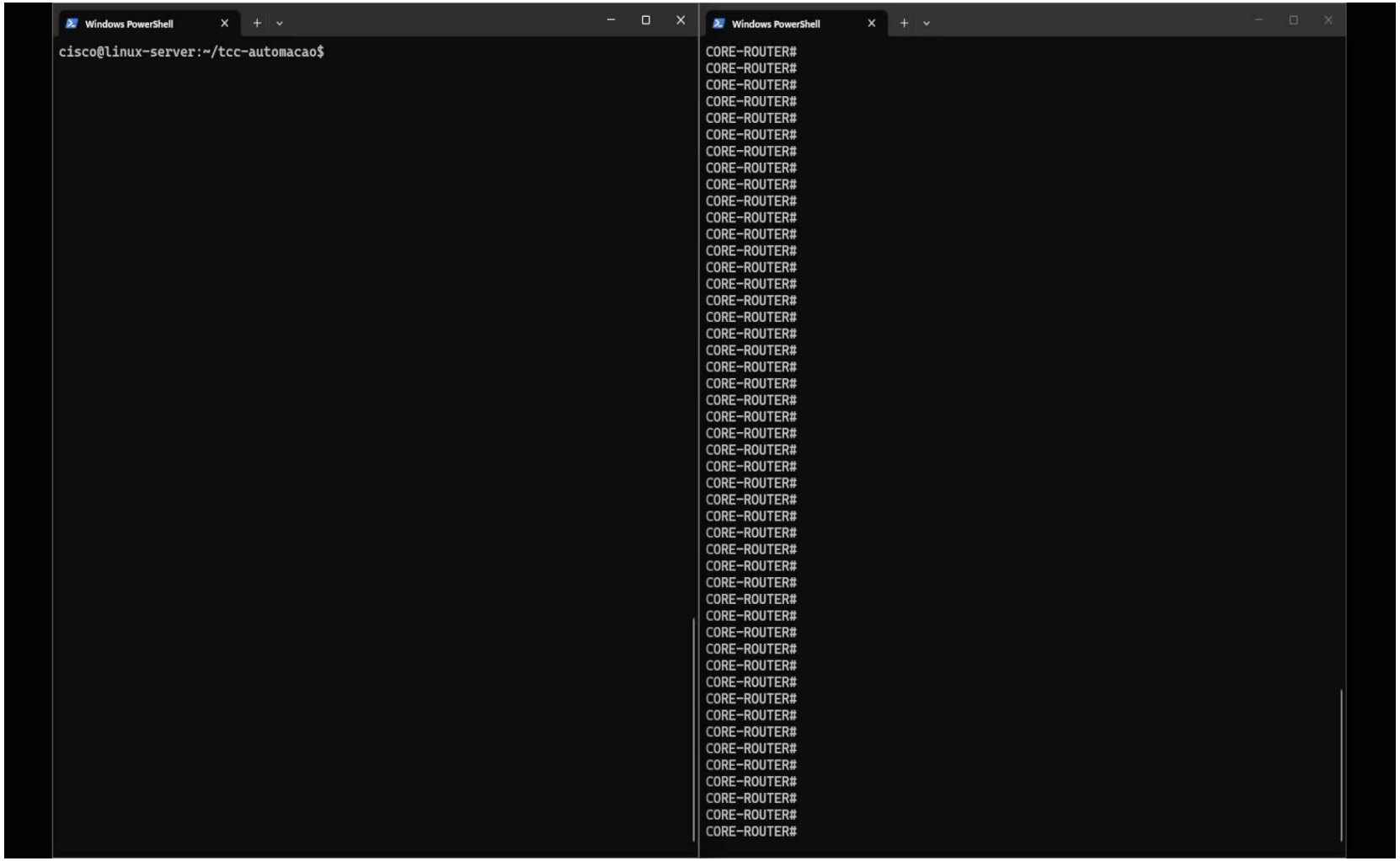


Decisão vem do YAML
roteador de contingência cadastrado, não escolhido sob pressão



Nada pela metade
nenhum cliente com configuração parcial

Demonstração: do YAML ao Route Reflector



1. Dados do cliente no YAML
2. Modelo Jinja2
3. Ativação com verificação
4. Prova no core (RR)
5. Pré-check: REJEITADO
6. Auditoria dos 4 roteadores
7. Limpeza do cliente

Discussão e limitações

Escala do laboratório

CML Free com 5 nós: sem roteador do cliente nem AS externo; imagens virtuais, não equipamentos de alta capacidade.

Um único operador

Quem mediu o manual também fez a automação: é o melhor caso do manual, e o ganho medido tende a ser conservador.

Regras e cenários restritos

8 regras no D4 (C04 e C05 não injetadas, para não cortar o acesso); D5 com um único cenário de falha.

Onde está o tempo

A configuração leva ~3 s; o resto é esperar a rede convergir e confirmar o serviço.

O ganho vai além da velocidade

Padronização

A configuração vem dos dados: erro de digitação ou de equipamento deixa de ser possível.

Verificação sistemática

Nada é dado como concluído sem prova no Route Reflector.

Rollback

Se a verificação falha, a mudança é desfeita sozinha.

Para o PROVEDOR

1. Auditoria (só leitura)
2. Ativação com rollback
3. Gestão de mudanças (Git)

Próximos passos

- **Nornir**
- **NetBox**
- **NETCONF/YANG**
- **Integração contínua**
- **MPLS e eBGP**

PERGUNTAS

Obrigado

Gabriel Henrique de Salles · salles.gabriel@pucpr.edu.br

Apoio: alternativas consideradas

Alternativa	O que é	Por que não foi a base
Ansible	Orquestrador declarativo, sem agente, playbooks em YAML	Esconde as etapas em módulos; verificação e rollback ficam menos explícitos e difíceis de medir
Nornir	Framework Python de orquestração, com paralelismo	Usa Netmiko e NAPALM como plugins; ganho pequeno com 4 roteadores. É a evolução natural
Cisco NSO	Plataforma comercial orientada a modelos, com transações	Licença e porte incompatíveis com o escopo de custo zero
NETCONF/YANG	Configuração por modelos de dados estruturados	Exige suporte e habilitação nos equipamentos; o cenário é de parque existente via CLI

Critérios: custo zero e código aberto · funciona via SSH no parque existente · comunidade ativa · cada etapa visível e mensurável.

Apoio: como o Jinja2 gera a configuração

templates/cliente_ativar.j2

```
interface Loopback{{ c.id }}
 description CLIENTE:{{ c.nome }} |
CIRCUITO:{{ c.circuito }} | VEL:{{
c.velocidade }}
 ip address {{ gateway }} {{ mascara }}
 no shutdown
router bgp {{ asn }}
 network {{ rede }} mask {{ mascara }}
```

Configuração gerada (cliente 101)

```
interface Loopback101
 description CLIENTE:EMPRESA-ALFA |
CIRCUITO:CORP-0101 | VEL:100M
 ip address 172.16.101.1 255.255.255.0
 no shutdown
router bgp 65001
 network 172.16.101.0 mask 255.255.255.0
```

id → Loopback · nome, circuito e velocidade → description · prefixo → ip address e network (gateway e máscara calculados pelo script).

Apoio: por que Route Reflector e por que verificar nele

Regra do iBGP

Rota aprendida de um vizinho iBGP não é repassada a outro vizinho iBGP.

Sem RR: malha completa, n(n−1)/2 sessões (4 roteadores = 6; 100 = 4.950).

Com RR: cada borda mantém uma única sessão com o core.

Sessões entre loopbacks, alcançáveis pelo OSPF mesmo se um enlace cair.

Verificação no RR

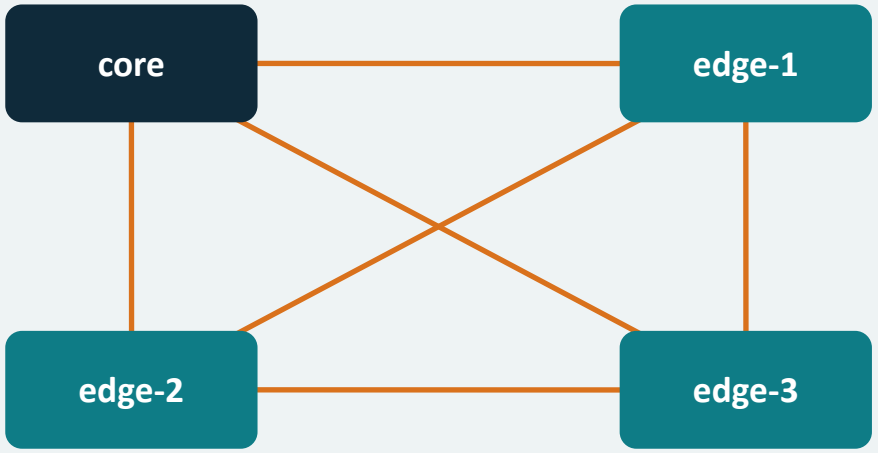
Se o prefixo está no BGP do core, a ativação se propagou pela rede, e não só foi aceita pela borda.

Ping ao gateway a partir do core: ≥ 80% de sucesso.

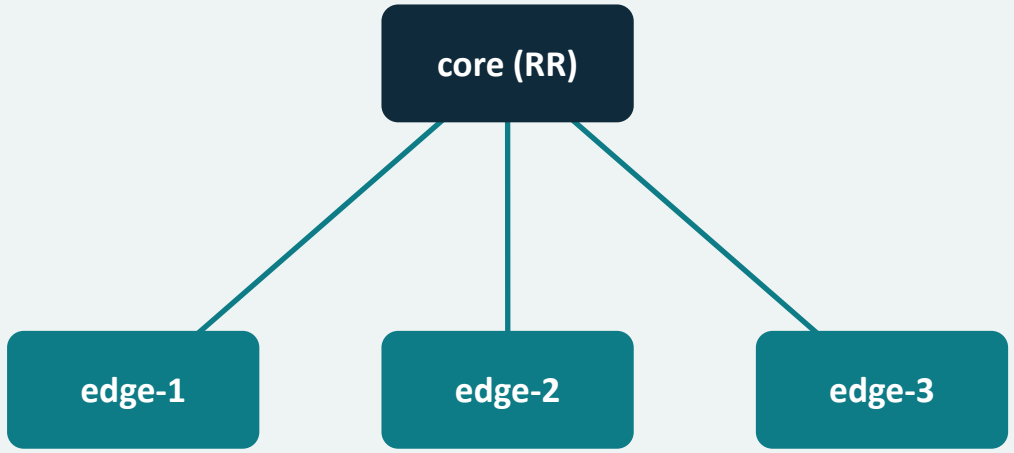
Consulta a cada 3 s, por até 45 s, antes de decidir entre salvar (write memory) e rollback.

Apoio: BGP, iBGP e Route Reflector

Sem RR: malha completa 6 sessões · 100 roteadores = 4.950



Com RR no core 3 sessões · 100 roteadores = 99



Regra do iBGP: rota aprendida de um vizinho iBGP não é repassada a outro. O RR é autorizado a refletir.

OSPF · infraestrutura
Leva só loopbacks e enlaces. Mantém as sessões iBGP de pé, mesmo se um cabo cair.

iBGP · clientes no AS 65001
A borda anuncia o prefixo (network) e o RR distribui para todas as bordas.

Core como RR
Centro da topologia, sem clientes e com visão completa: por isso a verificação é nele. Produção: 2 RRs.

Apoio: segurança e rastreabilidade

Credenciais fora do código

Lidas das variáveis LAB_USER e LAB_PASS; nenhuma senha em script ou arquivo.

Repositório privado

GitHub privado até a banca; sem backups nem YAML do lab, que têm hash de senha.

Horário único (NTP)

Todos sincronizam com o core: registros de equipamentos diferentes são comparáveis.

Registro de cada comando

Horário, usuário e forma de acesso (console ou SSH), mais os logs CSV dos scripts.